

Fast Food Management System

Project Documentation

Developed by: Maisam Taqi

Language: C++

Technologies:

File Handling, Functions, Console Application

GitHub Repository:

<https://github.com/maisam-taqi/fast-food-management-system-cpp.git>

Table of Contents:

1. Abstract
2. Planning and Research
3. Design
4. Development
5. Testing
6. Deployment

1. Abstract:

The **Fast-Food Management System (FFMS)** is designed to automate the management of a fast food restaurant's operations. The system enables employees to manage customer orders, staff records, food menus, and other restaurant-related functions like viewing and updating menu items, adding, editing, and deleting employee details. The primary goal of the system is to enhance the restaurant's efficiency by reducing human errors and streamlining food service operations.

2. Planning and Research:

Problem Statement:

Traditional fast food management systems often rely on manual operations, which can lead to inefficiencies, human errors, and difficulty in managing multiple orders simultaneously. The goal is to replace these manual processes with an automated system that improves the speed and accuracy of the service.

Objective Definition:

The **Fast-Food Management System** aims to simplify restaurant operations by managing customer orders, food menus, and staff details. The system will automate tasks such as adding, viewing, editing, and deleting food items and employee records, making operations more efficient and reducing errors.

Features Identification:

The core functionalities of the **Fast-Food Management System** include:

- **Login System:** Secure access for authorized staff members.
- **Order Management:** Add, view, and delete customer orders.

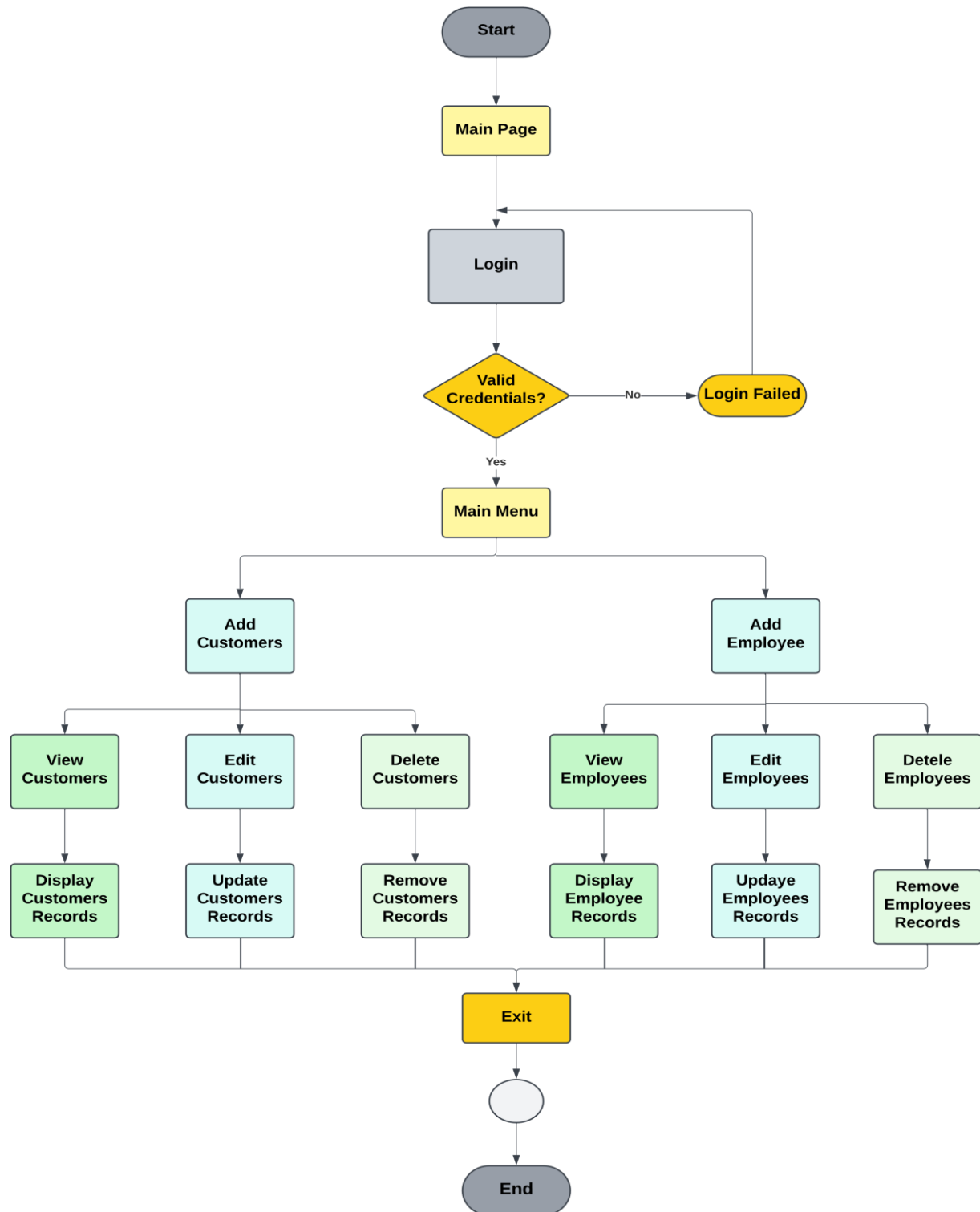
- **Menu Management:** Add, view, edit, and delete menu items.
- **Staff Record Management:** Add, view, edit, and delete staff details.
- **Exit Functionality:** Securely exit the system after use.

3. Design:

Overall Algorithm:

1. Algorithm (Summary)
2. Start the program and display the Welcome Page
3. Login Validation:
4. Prompt the user for username and password.
5. Allow up to 3 attempts to log in.
6. If successful, proceed to the main menu; otherwise, exit.
7. Display Main Menu:
8. Show the list of options:
9. Add Customer
10. Add Employee
11. View Customers
12. View Employees
13. Edit Customer
14. Edit Employee
15. Delete Customer
16. Delete Employee
17. Exit
18. User Choice Handling:
19. Based on the user's selection, call the corresponding function:
20. Add: Input and save customer/employee details to respective files.
21. View: Read and display customer/employee details from files.
22. Edit: Search, modify, and update customer/employee details.
23. Delete: Search and remove customer/employee details.
24. If invalid input, prompt to re-enter.
25. Exit:
26. On choosing "Exit," display a thank-you message and terminate the program.

Overall Flowchart:



Main Function Algorithm:

Void mainpage () – Function

1.0 Start

2.0 Display “|Fast Food Management System|”

3.0 Display “Prepared by: Hasnain Raza, Aqeel Iftikhar, Maisam Taqi”

4.0 Display "Subject: Programming Fundamentals “

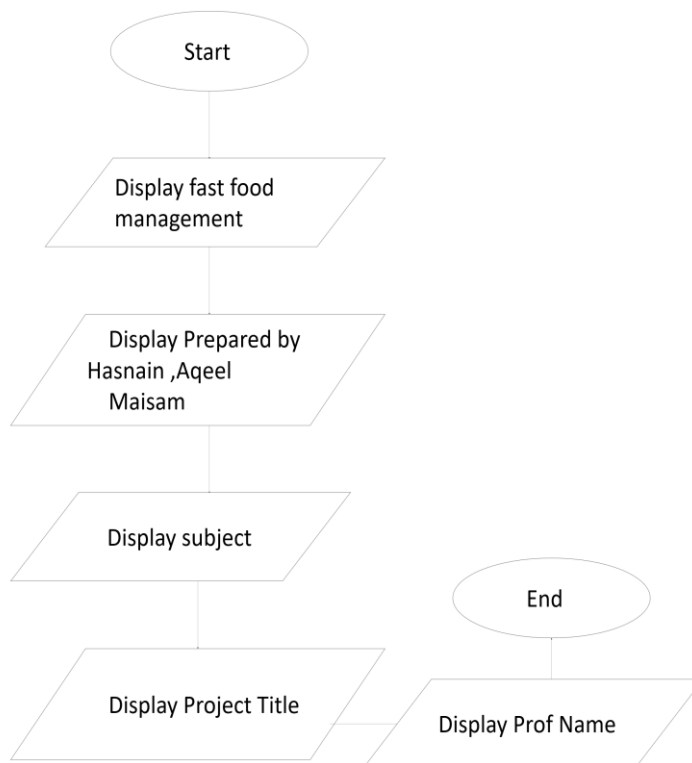
5.0 Display "Project title: Fast Fast Management System “

6.0 Display "Professor Name: Waqar Hussain “

7.0 Display "Government College university Faisalabad - Chiniot campus “

8.0 Endfunction.

Main Function Flowchart:



Login Function Algorithm:

Int login () – function

1.0 Start

2.0 Declare username, password

3.0 Initialize attempts = 0

4.0 Constant declare max_attempts, correct_username, correct_password

5.0 while (attempts < 3)

5.1 Input username, password

5.2 If (username == correct_username && password == correct_password)

5.2.1 Display “You are accessed to the System!!”

5.2.2 Return true

5.3 else

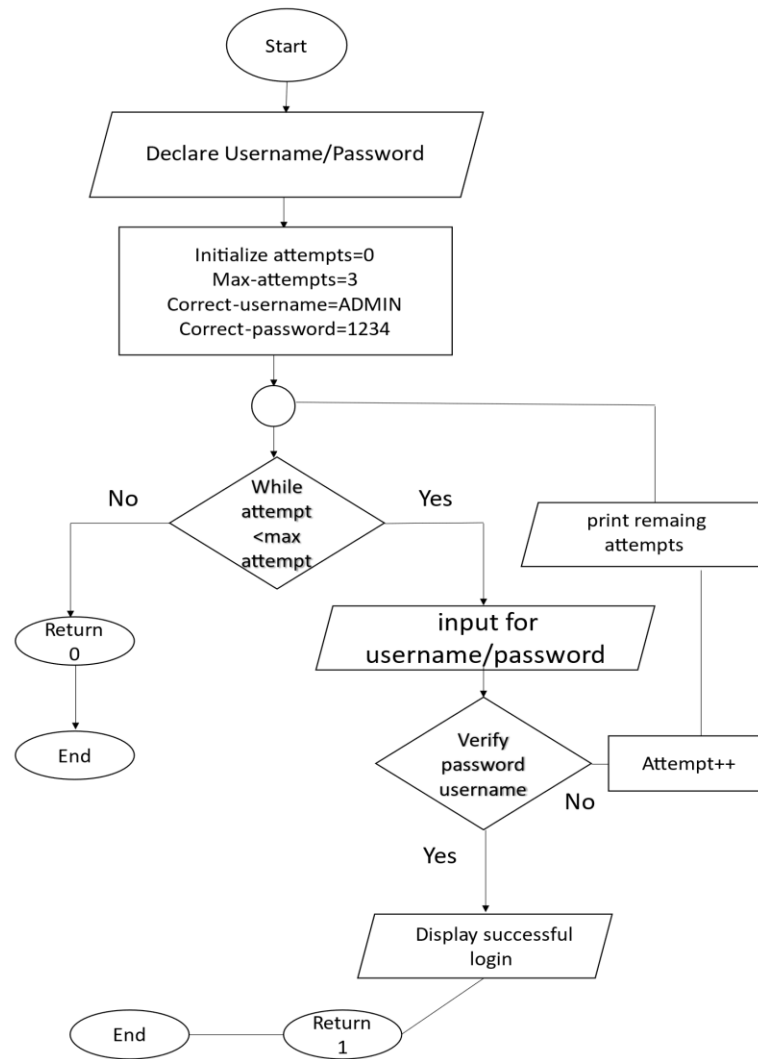
5.3.1 attempts ++

5.3.2 Display “No. of attempts remaining: “3 – attempts

6.0 return 0

7.0 Endfunction

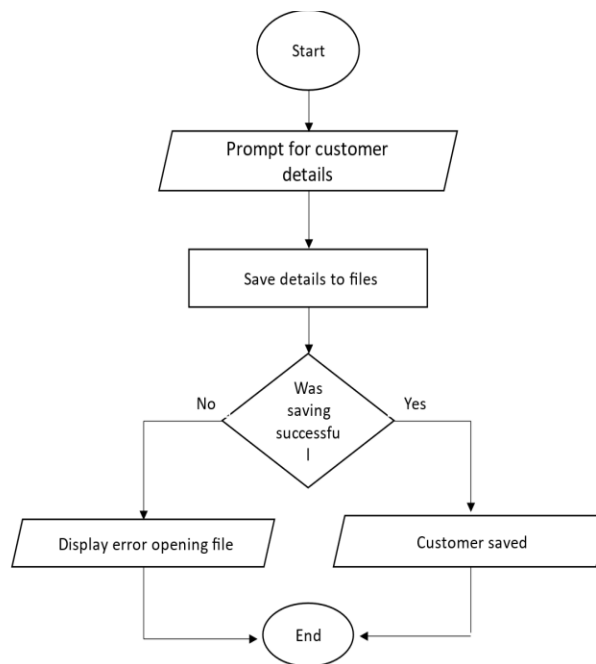
Flowchart:



Add customer Function Algorithm:

1. Start
2. Clear the screen.
3. Ask for customer name, address, order, and specialRequest.
4. Save the customer data in the file using the saveCustomerRecord function.
5. Print "Customer added successfully."
6. End

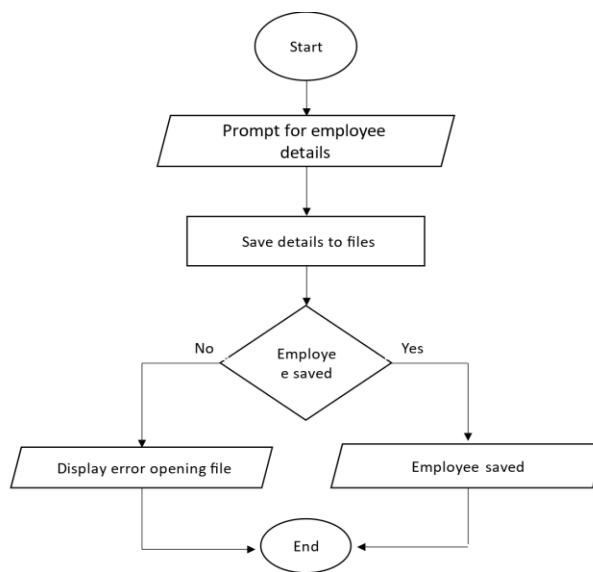
Flowchart:



Add employee Function Algorithm:

1. Start
2. Clear the screen.
3. Ask for employee ID, name, position, and assignedShift.
4. Save the employee data in the file using the saveEmployeeRecord function.
5. Print "Employee added successfully."
6. End

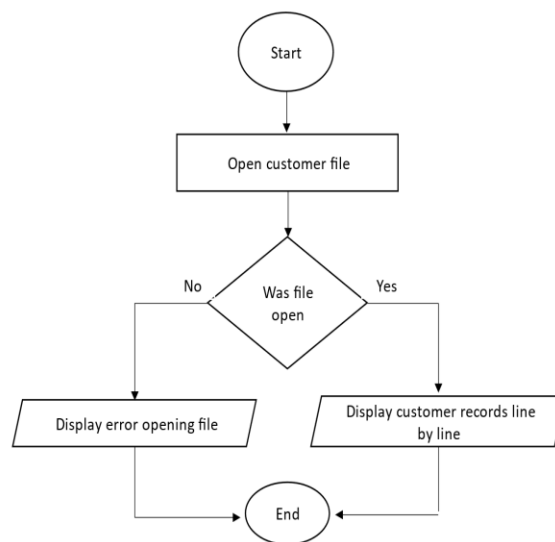
Flowchart:



View customer Function Algorithm:

1. Start
2. Clear the screen.
3. Open customer_orders.txt file.
4. If the file is available, read and display the data.
5. If not, print "Error: Could not open file."
6. Close the file.
7. End

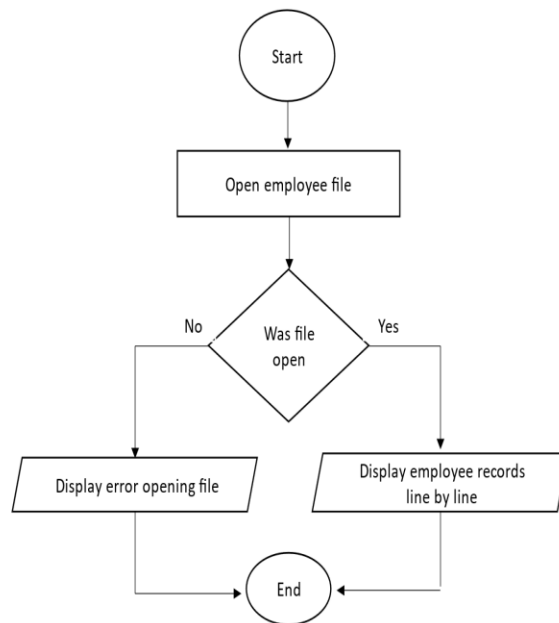
Flowchart:



View employees Function Algorithm:

1. Start
2. Clear the screen.
3. Open employee_records.txt file.
4. If the file is available, read and display the data.
5. If not, print "Error: Could not open file."
6. Close the file.
7. End

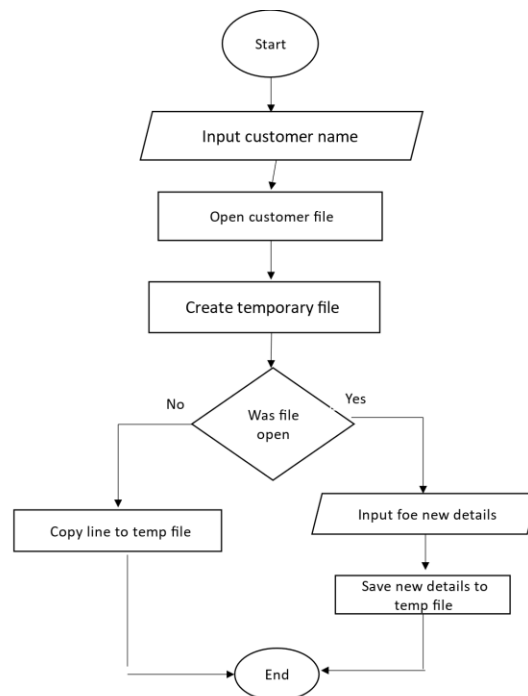
Flowchart:



Edit customer Function Algorithm:

1. Start
2. Clear the screen.
3. Ask for the name of the customer to edit.
4. Open the customer_orders.txt file for reading.
5. Create a temporary file temp_customer_orders.txt for writing.
6. For each line in the file:
 - If the line matches the customer name:
 - Ask for new name, address, order, and special Request.
 - Write updated details to the temporary file.
 - Otherwise, copy the original line to the temp file.
7. Replace the original file with the temporary file if the customer was found.
8. If not found, print "Customer not found."
9. End

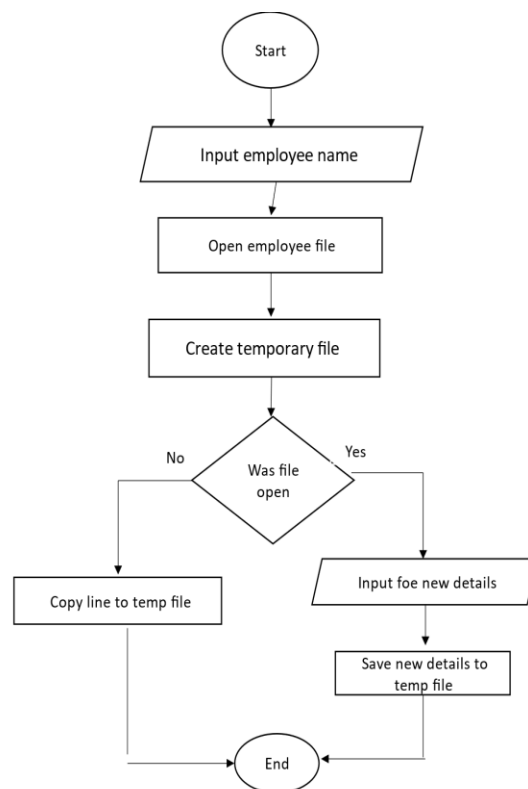
Flowchart:



Edit employees Function Algorithm:

1. Start
2. Clear the screen.
3. Ask for the employeeID to edit.
4. Open the employee_records.txt file for reading.
5. Create a temporary file temp_employee_records.txt for writing.
6. For each line in the file:
 - If the line matches the employeeID:
 - Ask for new employeeID, name, position, and assignedShift.
 - Write updated details to the temporary file.
 - Otherwise, copy the original line to the temp file.
7. Replace the original file with the temporary file if the employee was found.
8. If not found, print "Employee not found."
9. End

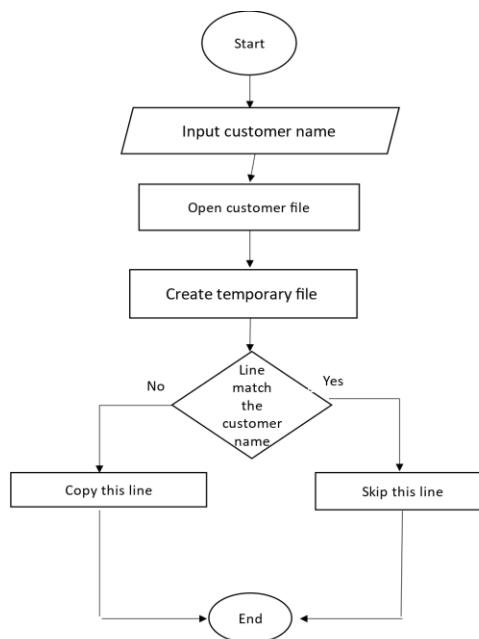
Flowchart:



Delete customer Function Algorithm:

1. Start
2. Clear the screen.
3. Ask for the name of the customer to delete.
4. Open the customer_orders.txt file for reading.
5. Create a temporary file temp_customer_orders.txt for writing.
6. For each line in the file:
 - If the line matches the customer name, skip it.
 - Otherwise, copy it to the temp file.
7. Replace the original file with the temporary file if the customer was found.
8. If not found, print "Customer not found."
9. End

Flowchart:



Development:

C++ Code:

- `#include <iostream>`
- `#include <string>`
- `#include <fstream>`
- `#include <conio.h> // For getch()`
- `#include <sstream> // For stringstream`
- `using namespace std;`

- `// Function Declarations`
- `int login();`
- `void mainPage();`
- `void callExit();`
- `void addCustomer();`
- `void addEmployee();`
- `void viewCustomers();`
- `void viewEmployees();`
- `void editCustomer();`
- `void editEmployee();`
- `void deleteCustomer();`
- `void deleteEmployee();`
- `void clearScreen();`
- `void saveCustomerRecord(const string& name, const string& address, const string& order, const string& specialRequest);`
- `void saveEmployeeRecord(const string& employeeID, const string& name, const string& position, const string& assignedShift);`
- `bool searchCustomerByName(const string& name);`
- `bool searchEmployeeByID(const string& employeeID);`

- `// Clear screen (cross-platform)`
- `void clearScreen()`
- `{`

- system("cls");
- }

- // Login Function
- int login()
- {
- clearScreen();
- string username, password;
- int attempts = 0;
- const int max_attempts = 3;
- const string correct_username = "pakistan";
- const string correct_password = "1234";

- while (attempts < max_attempts)
- {
- password.clear(); // Clear password for each attempt
- cout << "\n\n\n\n\n\n\n";
- cout << "\t\t\t\t\t ----- \n";
- cout << "\t\t\t\t\t | FAST FOOD MANAGEMENT SYSTEM | \n";
- cout << "\t\t\t\t\t ----- \n";
- cout << "\n\n";
- cout << "\t\t\t\t\tUsername: ";
- cin >> username;
- cout << "\t\t\t\t\tPassword: ";
- char ch;
- while ((ch = getch()) != '\r')
- { // '\r' is Enter key
 - if (ch == '\b' && !password.empty())
 - {
 - cout << "\b \b";
 - password.pop_back();
 - }
 - else if (ch != '\b')
 - {
 - cout << "*";

- password += ch;
 - }
- }
- if (username == correct_username && password == correct_password)
- {
 - cout << "\n\n";
 - cout << "\t\t\t\tYou have successfully logged into the system!\n\n";
 - return 1;
- }
- else
- {
 - attempts++;
 - cout << "\n\n\n\n\n\n";
 - cout << "\t\t\t\tNo. of attempts remaining: " << max_attempts - attempts << "\n\n";
- }
- }
- return 0;
- }

- // Main Page Function
- void mainPage() {
- system("cls");
- cout << "\t\t\t\t\t" << __DATE__ << endl;
- cout << "\n";
- cout << "\t\t\t\t\t ----- \n";
- cout << "\t\t\t\t\t | FAST FOOD MANAGEMENT SYSTEM | \n";
- cout << "\t\t\t\t\t ----- \n";
- cout << "\n\n";

- `cout << "\t\t\tPrepared By :Maisam taqi\n";`
 - `cout << "\t\t\tSubject : Programming Fundamentals\n";`
 - `cout << "\t\t\tProject Title : Fast Food Management System\n";`
 - `cout << "\t\t\tProfessor Name : Waqar Hussain\n";`
 - `cout << "\t\t\tGovernment College University Faislabad Chiniot Campus\n";`
 - `cout << "\n\n";`
 - `system("pause");`
 - `}`
 - `// Exit function`
 - `void callExit()`
 - `{`
 - `system("cls");`
 - `cout << "\n\n\n\n\n";`
 - `cout << "\n\tTaking You Out of The System\n";`
 - `cout << "\n\n\n\n\n\n\n\n\n\n";`
 - `cout << "\t\t\tYou are out of the System.\n";`
 - `cout << "\t\t\t\tThank You!\n";`
 - `exit(0);`
 - `}`
-
- `// Save Customer Record`
 - `void saveCustomerRecord(const string& name, const string& address, const string& order, const string& specialRequest) {`
 - `ofstream file("customer_orders.txt", ios::app); // Open file in append mode`
 - `if (file.is_open())`
 - `{`
 - `file << name << "\n";`
 - `file << address << "\n";`
 - `file << order << "\n";`
 - `file << specialRequest << "\n";`
 - `file << "-----\n";`
 - `file.close();`
 - `cout << "Customer details saved successfully." << endl;`
 - `}`

- else
- {
- cout << "Error: Unable to open file!" << endl;
- }
- }

- // Save Employee Record
- void saveEmployeeRecord(const string& employeeID, const string& name, const string& position, const string& assignedShift) {
- ofstream file("employee_records.txt", ios::app); // Append mode
- if (file.is_open())
- {
- file << employeeID << "\n";
- file << name << "\n";
- file << position << "\n";
- file << assignedShift << "\n";
- file << "-----\n";
- file.close();
- cout << "Employee details saved successfully." << endl;
- }
- else
- {
- cout << "Error: Unable to open file!" << endl;
- }
- }
- // Add Customer Function
- void addCustomer()
- {
- clearScreen();
- string name, address, order, specialRequest;
- cin.ignore(); // To clear any leftover input buffer
- cout << "Enter the name of the customer: ";
- getline(cin, name);
- cout << "Enter the address of the customer: ";
- getline(cin, address);
- cout << "Enter the order (Burger, Fries, Drink,): ";

- `getline(cin, order);`
 - `cout << "Enter any special requests: ";`
 - `getline(cin, specialRequest);`
-
- `saveCustomerRecord(name, address, order, specialRequest);`
 - `}`
 - `// Add Employee Function`
 - `void addEmployee()`
 - `{`
 - `clearScreen();`
 - `string employeeID, name, position, assignedShift;`
 - `cin.ignore(); // To clear any leftover input buffer`
 - `cout << "Enter Employee ID: ";`
 - `getline(cin, employeeID);`
 - `cout << "Enter Employee Name: ";`
 - `getline(cin, name);`
 - `cout << "Enter Employee Position: ";`
 - `getline(cin, position);`
 - `cout << "Enter Assigned Shift: ";`
 - `getline(cin, assignedShift);`
-
- `saveEmployeeRecord(employeeID, name, position, assignedShift);`
 - `}`
-
- `// View Customers Function`
 - `void viewCustomers()`
 - `{`
 - `clearScreen();`
 - `ifstream file("customer_orders.txt");`
 - `string line;`
 - `if (file.is_open())`
 - `{`
 - `cout << "Customer Orders:\n";`

- cout << "-----\n";
- while (getline(file, line))
- {
 - cout << line << endl;
- }
- file.close();
- }
- else
- {
- cout << "Error: Unable to open customer orders file!" << endl;
- }
- }

- // View Employees Function
- void viewEmployees()
- {
- clearScreen();
- ifstream file("employee_records.txt");
- string line;
- if (file.is_open())
- {
- cout << "Employee Records:\n";
- cout << "-----\n";
- while (getline(file, line))
- {
 - cout << line << endl;
- }
- file.close();
- }
- else
- {
- cout << "Error: Unable to open employee records file!" << endl;
- }
- }

- // Edit Customer Function
- void editCustomer()
- {
- clearScreen();
- string name, line;
- cout << "Enter the name of the customer to edit: ";
- cin.ignore(); // To clear any leftover input buffer
- getline(cin, name);

- ifstream file("customer_orders.txt");
- ofstream tempFile("temp_customer_orders.txt");
- bool found = false;

- while (getline(file, line))
- {
- if (line == name)
- {
 - found = true;
 - cout << "Editing details for customer: " << name << endl;
 - string newName, newAddress, newOrder, newSpecialRequest;
 - cout << "Enter new name: ";
 - getline(cin, newName);
 - cout << "Enter new address: ";
 - getline(cin, newAddress);
 - cout << "Enter new order (e.g., Burger, Fries): ";
 - getline(cin, newOrder);
 - cout << "Enter new special requests: ";
 - getline(cin, newSpecialRequest);

 - saveCustomerRecord(newName, newAddress, newOrder, newSpecialRequest);
 - continue; // Skip writing the old record
- }
- tempFile << line << "\n";

- }

- if (found)
- {
- cout << "Customer details updated successfully." << endl;
- file.close();
- tempFile.close();
- remove("customer_orders.txt");
- rename("temp_customer_orders.txt", "customer_orders.txt");
- }
- else
- {
- cout << "Customer not found!" << endl;
- }
- }

- // Edit Employee Function
- void editEmployee()
- {
- clearScreen();
- string employeeID, line;
- cout << "Enter the Employee ID to edit: ";
- cin.ignore(); // To clear any leftover input buffer
- getline(cin, employeeID);

- ifstream file("employee_records.txt");
- ofstream tempFile("temp_employee_records.txt");
- bool found = false;

- while (getline(file, line))
- {

- if (line == employeeID)
- {
 - found = true;
 - cout << "Editing details for Employee ID: " << employeeID << endl;
 - string newEmployeeID, newName, newPosition, newAssignedShift;
 - cout << "Enter new Employee ID: ";
 - getline(cin, newEmployeeID);
 - cout << "Enter new Name: ";
 - getline(cin, newName);
 - cout << "Enter new Position: ";
 - getline(cin, newPosition);
 - cout << "Enter new Assigned Shift: ";
 - getline(cin, newAssignedShift);
 - saveEmployeeRecord(newEmployeeID, newName, newPosition, newAssignedShift);
 - continue; // Skip writing the old record
- }
- tempFile << line << "\n";
- }

- if (found)
- {
 - cout << "Employee details updated successfully." << endl;
 - file.close();
 - tempFile.close();
 - remove("employee_records.txt");
 - rename("temp_employee_records.txt", "employee_records.txt");
- }
- else
- {
 - cout << "Employee not found!" << endl;
- }

- }

- // Delete Customer Function
- void deleteCustomer()
- {
- clearScreen();
- string name, line;
- cout << "Enter the name of the customer to delete: ";
- cin.ignore(); // To clear any leftover input buffer
- getline(cin, name);

- ifstream file("customer_orders.txt");
- ofstream tempFile("temp_customer_orders.txt");
- bool found = false;

- while (getline(file, line))
- {
- if (line == name)
- {
- o found = true;
- o continue; // Skip deleting this record
- }
- tempFile << line << "\n";
- }

- if (found)
- {
- cout << "Customer deleted successfully." << endl;
- file.close();
- tempFile.close();
- remove("customer_orders.txt");

- rename("temp_customer_orders.txt", "customer_orders.txt");
- }
- else
- {
- cout << "Customer not found!" << endl;
- }
- }

- // Delete Employee Function
- void deleteEmployee()
- {
- clearScreen();
- string employeeID, line;
- cout << "Enter the Employee ID to delete: ";
- cin.ignore(); // To clear any leftover input buffer
- getline(cin, employeeID);

- ifstream file("employee_records.txt");
- ofstream tempFile("temp_employee_records.txt");
- bool found = false;

- while (getline(file, line))
- {
- if (line == employeeID)
- {
 - found = true;
 - continue; // Skip deleting this record
- }
- tempFile << line << "\n";
- }

- if (found)
- {
- cout << "Employee deleted successfully." << endl;
- file.close();
- tempFile.close();
- remove("employee_records.txt");
- rename("temp_employee_records.txt", "employee_records.txt");
- }
- else
- {
- cout << "Employee not found!" << endl;
- }
- }

- // Main function
- int main()
- {
- cout << __DATE__ << endl;
- mainPage();
- system("pause");

- if (login())
- {
- char choice;
- do
- {
 - clearScreen();
 - cout << "Fast Food Management System\n";
 - cout << "=====\n";
 - cout << "1. Add Customer\n";
 - cout << "2. Add Employee\n";
 - cout << "3. View Customers\n";
 - cout << "4. View Employees\n";
 - cout << "5. Edit Customer\n";
 - cout << "6. Edit Employee\n";

- cout << "7. Delete Customer\n";
- cout << "8. Delete Employee\n";
- cout << "9. Exit\n";
- cout << "Enter your choice: ";
- cin >> choice;

- switch (choice)
- {
 - case '1': addCustomer(); break;
 - case '2': addEmployee(); break;
 - case '3': viewCustomers(); break;
 - case '4': viewEmployees(); break;
 - case '5': editCustomer(); break;
 - case '6': editEmployee(); break;
 - case '7': deleteCustomer(); break;
 - case '8': deleteEmployee(); break;
 - case '9': callExit(); break;
 - default: cout << "Invalid choice! Try again.\n";
- }
- cout << "\nPress any key to return to menu...\n";
- system("pause");
- } while (choice != '9');
- }
- callExit();
- return 0;
- }

Testing:

FAST FOOD MANAGEMENT SYSTEM

Prepared By : Maisam taqi

Subject : Programming Fundamentals

Project Title : Fast Food Management System

Professor Name : Waqar Hussain

Government College University Faislabad Chiniot Campus

Press any key to continue . . .

FAST FOOD MANAGEMENT SYSTEM

Username: pakistan

Password: ****

Fast Food Management System

=====

- i. Add Customer
- ii. Add Employee
- iii. View Customers
- iv. View Employees
- v. Edit Customer
- vi. Edit Employee
- vii. Delete Customer
- viii. Delete Employee
- ix. Exit
- x. Enter your choice

Add Customer:

- Enter the name of the customer: Ali
- Enter the address of the customer: Chiniot
- Enter the order (Burger, Fries, Drink,): Burger
- Enter any special requests: no
- Customer details saved successfully.
- Press any key to return to menu...
- Press any key to continue . . .

Add Employee:

- Enter Employee ID: 12345
- Enter Employee Name: Ahmed
- Enter Employee Position: front door
- Enter Assigned Shift: morning
- Employee details saved successfully.
- Press any key to return to menu...
- Press any key to continue . . .

View Customers:

- Ali
- Chiniot
- Burger
- no
- Press any key to return to menu...
- Press any key to continue . . .

View Employees:

- 12345
- Ahmed
- front door
- morning
- Press any key to return to menu...
- Press any key to continue . . .

Edit Customer:

- Enter the name of the customer to edit: Ali
- Editing details for customer: Ali
- Enter new name: Hassan
- Enter new address: Fsd
- Enter new order (e.g., Burger, Fries): fries
- Enter new special requests: yes
- Customer details saved successfully.
- Customer details updated successfully.
- Press any key to return to menu...
- Press any key to continue . . .

Edit Employee:

- Enter the Employee ID to edit: 12345
- Editing details for Employee ID: 12345
- Enter new Employee ID: 2233
- Enter new Name: hasnanin
- Enter new Position: back door
- Enter new Assigned Shift: night
- Employee details saved successfully.
- Employee details updated successfully.
- Press any key to return to menu...
- Press any key to continue . . .

Delete Customer:

- Enter the name of the customer to delete: Hassan
- Customer deleted successfully.
- Press any key to return to menu...
- Press any key to continue . . .

Delete Employee:

- Enter the Employee ID to delete: 2233
- Employee deleted successfully.
- Press any key to return to menu...
- Press any key to continue . . .

Exit:

- Taking You Out of The System
- You are out of the System.
- Thank You!

Deployment:

Once the system is tested and ready, it will be deployed in a fast-food restaurant environment. It will allow restaurant staff to easily manage food orders, maintain menu items, and keep track of employee records. After deployment, the system will be continuously monitored, and updates will be made to improve functionality as needed.

User Documentation for Fast Food Management System

Introduction:

The **Fast-Food Management System (FFMS)** is designed to assist restaurant staff in efficiently managing orders, menu items, and employee details. This guide provides step-by-step instructions for using the system.

1. Login Process:

Upon launching the system, you will be prompted to enter your username and password. The default credentials are:

- **Username:** admin
- **Password:** 1234

You have three attempts to enter the correct credentials. After three unsuccessful attempts, the system will exit.

2. Main Menu Navigation:

After logging in successfully, you will be directed to the main menu, which offers the following options:

1. Add Customer Order
2. Add Food Item to Menu
3. View Orders
4. View Menu Items

5. Edit Customer Order
6. Edit Menu Item
7. Delete Customer Order
8. Delete Menu Item
9. Exit

3. Adding a Customer Order:

- To add a new customer order, select option **1. Add Customer Order**.
- The system will prompt you to enter the following details:
 - Customer Name
 - Ordered Food Item(s)
 - Quantity
 - Total Price

4. Adding Food Item to Menu:

- To add a new food item, select option **2. Add Food Item to Menu**.
- The system will ask you to enter:
 - Food Item Name
 - Description
 - Price
 - Category (e.g., Burger, Drink, Dessert)

5. Viewing Orders

- To view the list of all orders, select **3. View Orders**.
- The system will display:
 - Order No.
 - Customer Name
 - Ordered Items
 - Quantity
 - Total Price

6. Viewing Menu Items:

- To view the list of all menu items, select **4. View Menu Items**.
- The system will display:
 - Food Item No.
 - Food Item Name
 - Description

- Price
- Category

7. Editing Customer Order:

- To edit an existing order, select **5. Edit Customer Order**.
- You will be prompted to enter the order number to edit. After selecting the order, you can modify details such as food items or quantities.

8. Editing Menu Item:

- To edit a menu item, select **6. Edit Menu Item**.
- You will be prompted to enter the food item number. After selecting the item, you can update its details like name, price, and description.

9. Deleting Customer Order:

- To delete a customer order, select **7. Delete Customer Order**.
- You will be prompted to enter the order number to delete.

10. Deleting Menu Item:

- To delete a menu item, select **8. Delete Menu Item**.
- You will be prompted to enter the food item number to delete.

11. Exiting the System:

- To log out and exit the system, select **9. Exit**.
- The system will log you out and display a message: "You are out of the System. Thank you!"

Maintenance:

After deployment, the system will be regularly monitored for any issues. The development team will release updates and patches to improve functionality, fix bugs, and add new features as required.

By converting the system from a hospital management system to a fast food management system, the focus has shifted to managing food orders, menu items, and staff data within the context of a restaurant environment, making it easier for staff to handle daily operations in a fast food establishment.